

LAB : *Retrieval-Augmented Generation* ou Génération Augmentée par Récupération

Table des matières

PARTIE 1 : Extraction de contenu d'un fichier PDF	2
Chargement et extraction de contenu d'un fichier PDF avec LangChain (PyPDFLoader)	2
Segmentation de texte pour préparation au RAG avec LangChain.....	2
Transformation des chunks en vecteurs sémantiques avec un modèle Hugging Face : Génération d'embeddings textuels	3
Création d'une base vectorielle en mémoire pour stockage et recherche d'embeddings	3
Indexation des documents dans la base vectorielle pour recherche sémantique.....	3
Recherche sémantique dans une base vectorielle pour retrouver les informations pertinentes	3
Agent RAG	3
PARTIE 2 : Extraction de contenu d'une base de données SQL (SQLite).....	4
Connexion à une base de données SQL (SQLite) avec LangChain	4
Création d'un tool personnalisé pour interroger une base SQL avec un agent	4
Création d'un agent LLM pour interroger une base SQL.....	5
Interrogation d'un agent SQL via langage naturel et récupération des résultats.....	5

PARTIE 1 : Extraction de contenu d'un fichier PDF

Chargement et extraction de contenu d'un fichier PDF avec LangChain (PyPDFLoader)

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("acmecorp-employee-handbook.pdf")

data = loader.load()

print(data)
```

Segmentation de texte pour préparation au RAG avec LangChain

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200, add_start_index=True
)
all_splits = text_splitter.split_documents(data)

print(len(all_splits))
```

Transformation des chunks en vecteurs sémantiques avec un modèle Hugging Face : Génération d'embeddings textuels

```
from langchain_community.embeddings import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
```

Création d'une base vectorielle en mémoire pour stockage et recherche d'embeddings

```
from langchain_core.vectorstores import InMemoryVectorStore

vector_store = InMemoryVectorStore(embeddings)
```

Indexation des documents dans la base vectorielle pour recherche sémantique

```
ids = vector_store.add_documents(documents=all_splits)
```

Recherche sémantique dans une base vectorielle pour retrouver les informations pertinentes

```
results = vector_store.similarity_search(
    "How many days of vacation does an employee get in their first year?"
)

print(results[0])
```

Agent RAG

```
from langchain.tools import tool

@tool
def search_handbook(query: str) -> str:
    results = vector_store.similarity_search(query)
    return results[0].page_content

from langchain.agents import create_agent
from langchain.messages import HumanMessage
```

```
from langchain_ollama import ChatOllama

# Initialiser le modèle Ollama
model = ChatOllama(
    model="llama3.2:3b", # ou mistral, gemma, etc.
    temperature=0
)

agent = create_agent(
    model=model,
    tools=[search_handbook],
    system_prompt="You are a helpful agent that can search the employee hand-
book for information."
)

response = agent.invoke(
    {"messages": [HumanMessage(content="How many days of vacation does an em-
ployee get in their first year?")]}
)
print(response['messages'][-1].content)
```

PARTIE 2 : Extraction de contenu d'une base de données SQL (SQLite)

Connexion à une base de données SQL (SQLite) avec LangChain

```
from langchain_community.utilities import SQLDatabase

db = SQLDatabase.from_uri("sqlite:///Chinook.db")
```

Création d'un tool personnalisé pour interroger une base SQL avec un agent

```
from langchain.tools import tool

@tool
def sql_query(query: str) -> str:

    """Obtain information from the database using SQL queries"""

    try:
        print(f"Executing SQL query: {query}")
        return db.run(query)

    except Exception as e:
        return f"Error: {e}"
```

```
sql_query.invoke("SELECT * FROM Artist LIMIT 10")
```

Création d'un agent LLM pour interroger une base SQL

```
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langchain_ollama import ChatOllama

# Initialiser le modèle Ollama
model = ChatOllama(
    model="llama3.2:3b", # ou mistral, gemma, etc.
)
system_prompt = """You are a SQL expert.

Rules:
- Only use sql_query tool
- The sql_query tool takes a SQL query as input and returns the result of the query.
- Only use available columns
- If information does not exist, say so
- Do not guess
- you have to return the results in a human readable format, do not return raw SQL results or a sql query.

Database schema:
Table Artist:
- ArtistId
- Name
"""

agent = create_agent(
    model=model,
    tools=[sql_query],
    system_prompt=system_prompt
)
```

Interrogation d'un agent SQL via langage naturel et récupération des résultats

```
question = HumanMessage(content="Give me the first 5 artists in the database")

response = agent.invoke(
    {"messages": [question]}
)
print(response['messages'][-1].content)
```